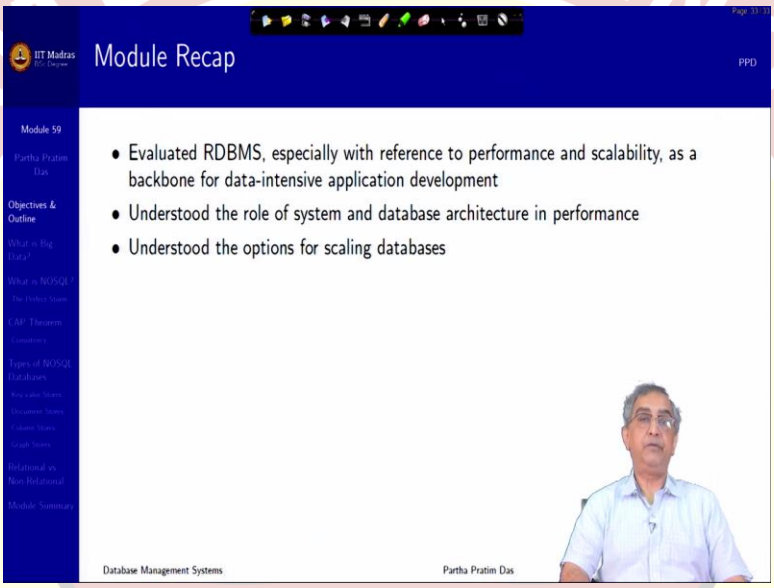
IIT Madras

ONLINE DEGREE

Database Management Systems
Professor Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology Kharagpur
Non-Relational DBMS: NOSQL

Welcome to Module 59, of database management systems course, in the IIT Madras online BSc program.

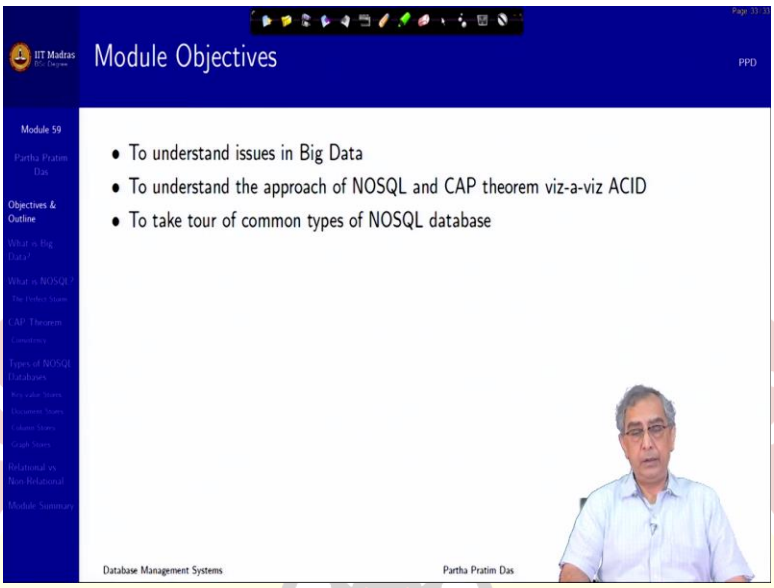
(Refer Slide Time: 00:26)



In the last module, we stepped back and took a view of the RDBMS with reference to performance and scalability. That what we have designed is something which is good robust in an RDBMS, but will it scale? Will it continue to perform at the same level? And in this connection, we took a quick tour of different system and database architecture.

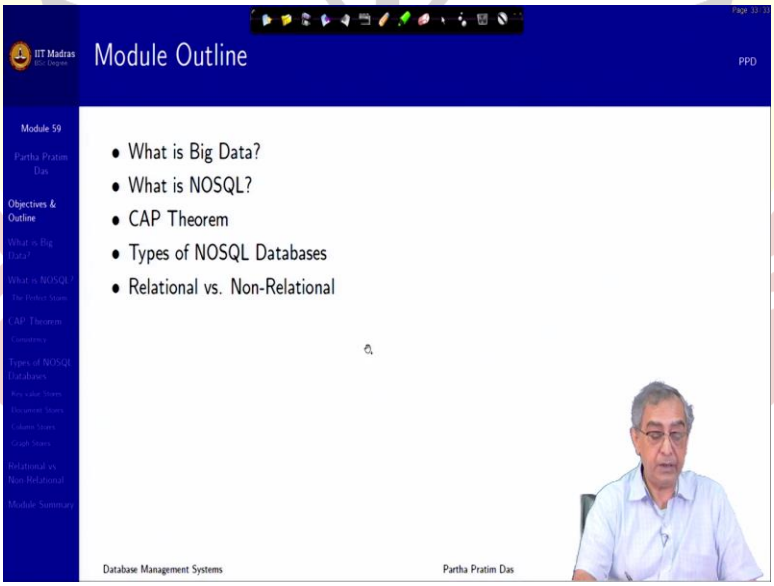
And kind of came to the conclusion that, it is a database distributed architecture, that is what we have to adhere to, if you are really looking for scaling high and high. And possibly scale out that is, horizontal scaling is what will be the option that we will have to mostly go with.

(Refer Slide Time: 01:18)



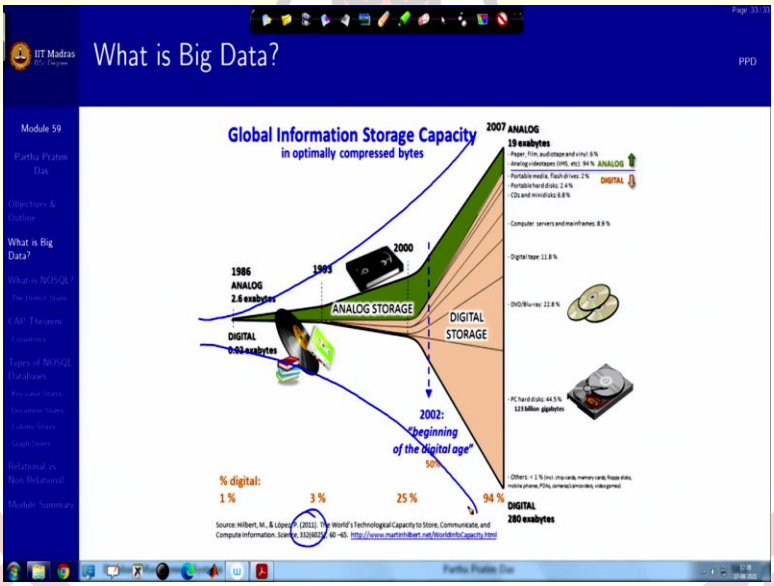
So, with that background, now I talk about something little more recent developments, that is to understand the issues in big data, you all have heard about big data, I am sure. And based on that, we will try to just take a quick look at the approach of what is called NOSQL. And assess the CAP theorem, viz a viz ACID. And take a quick tour of common types of NOSQL databases.

(Refer Slide Time: 01:52)



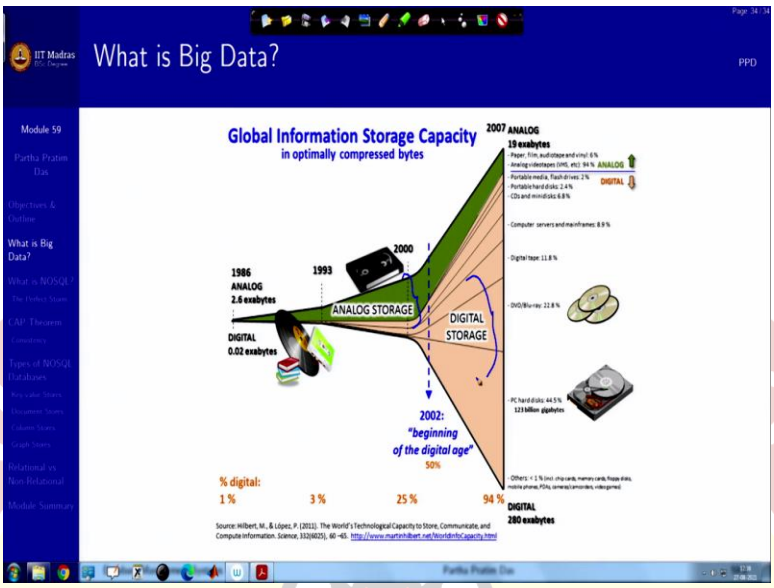
So, this is all that we discuss here.

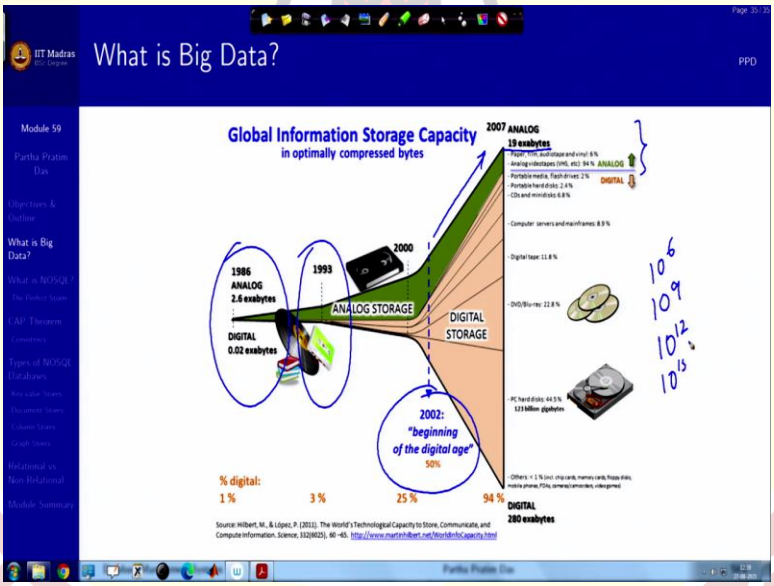
(Refer Slide Time: 01:56)



So, what is big data? Big data, as the name suggests, is big, it is, it is bigger than, much, much bigger than what we can conceive. So, this graph which itself is pretty old, because it was published in 2011. So, more than it, more than ten years old. You can see the basic trend, that how the data volume in the world is increasing.

(Refer Slide Time: 2:31)

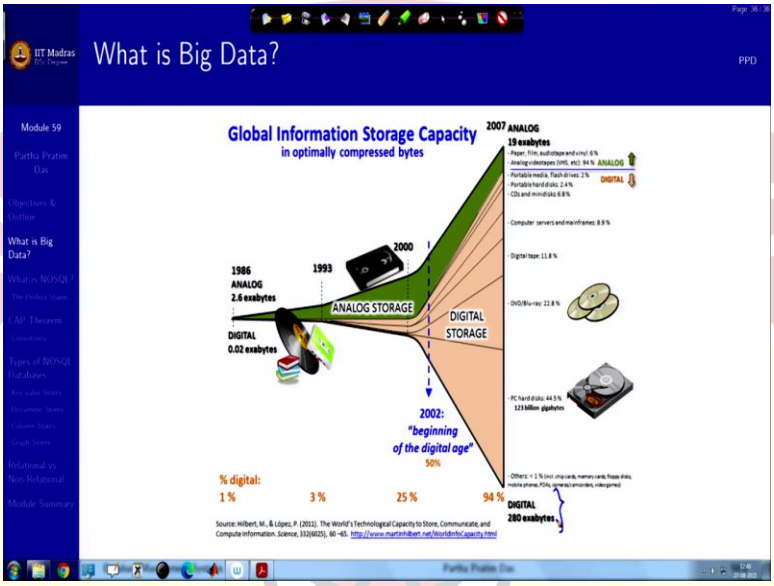




And it also shows the division between, analog storage and digital storage. So, you can see that, the while the analog storage is more or less remaining the same after the initial growth and is in fact possibly slowly reducing, the digital storage is the main explosion that is happening. So, this is the 80s where analog storage used to dominate, then we have introduction of lot of different digital storage is in the early 90s. And in 2002, it is typically called the beginning of the digital age where the analog and the digital storage started balancing and then exploding.

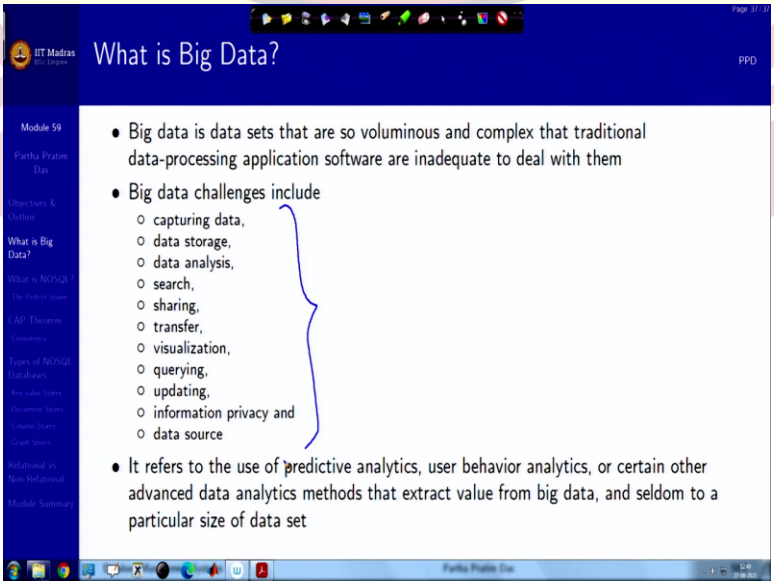
And you can see what is the explosion that is going on. And now in, in with reference to 2011 the estimate was there is 19 exabyte, you know what is exa? You have mega is 10 to the power 6, giga is 10 to the power 9. This is, then you have beta, then you have exa.

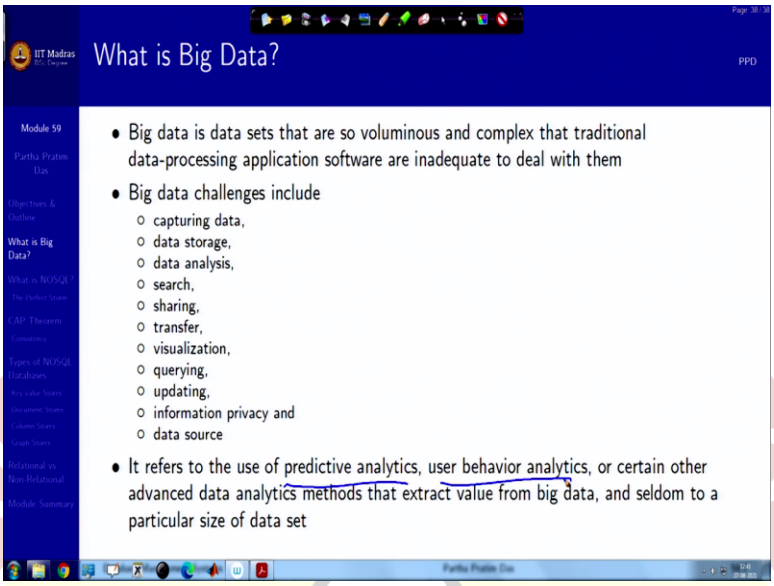
(Refer Slide Time: 03:43)



So, you have hugely large, hugely, hugely large of this, which is 19 exabytes, which mean analog means the paper copies, the micro fresh film and all that, analog device, recording devices. In contrast, the data digitally was estimated to be nearly 300 exabytes, in all different digital forms. And with the mobile devices, handle devices this is exploding like anything.

(Refer Slide Time: 04:13)

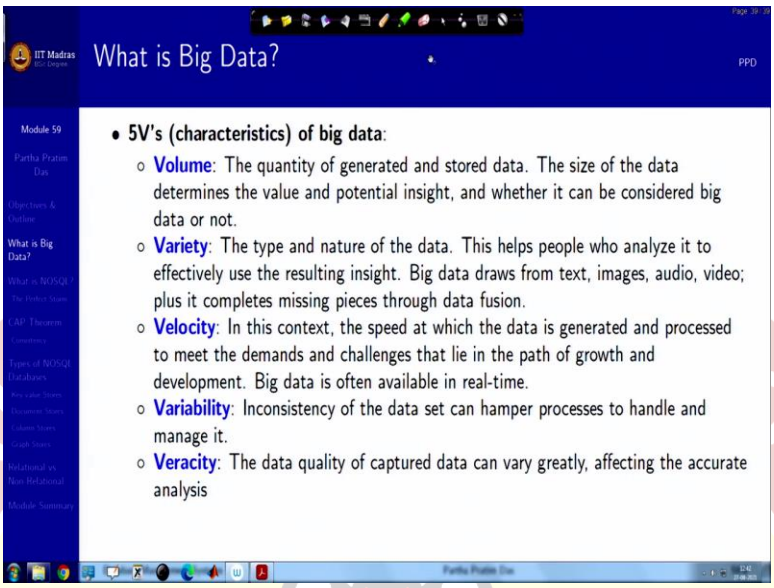




So, this is the basic first paradigm of big data, which is a data set is so voluminous and complex, that traditional data processing applications are no more adequate to deal with them. And there are several challenges that exists, like how you capture the data, how you store that data, forget about any centralized store, even a simple distributed store. How do you analyze the data, visualize that data? How do you query that data and so on so forth?

I mean, even a single social media application like Facebook is a humungous big data problem. And big data also is not just data which is big, but it refers to different kinds of novel analysis like predictive analysis, user behavior analysis, Amazon knows really what you are going to buy in the next one month, has a fair estimate of that, without you knowing it actually, clearly. So, these are, these are things that big data is leading us to.

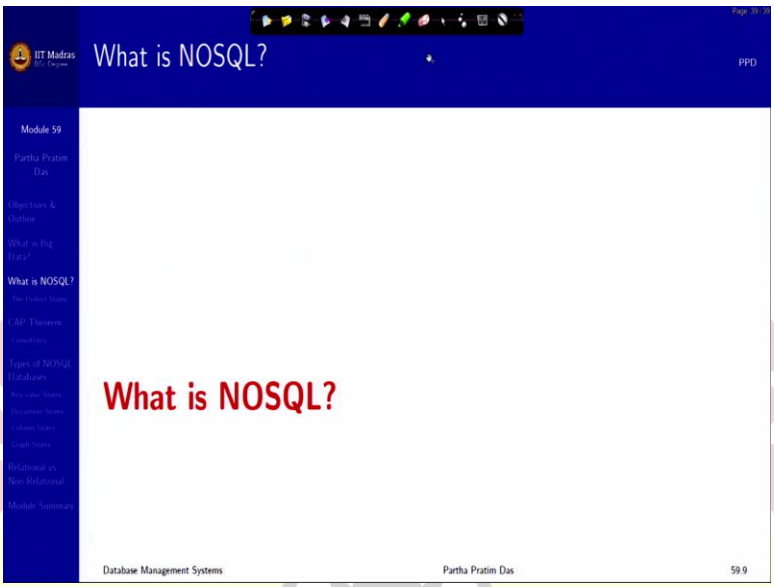
(Refer Slide Time: 05:09)



So, there are certain characteristics of big data, naturally volume is the first, variety is next. That is the type and nature of data is widely varied text image, audio, video everything. The velocity, the speed at which the data is generated and processed is increasing enormously. Variability, that the inconsistency of datasets can hamper, you can no more, no more assume that the data is all consistent. There will be lot of input data which are not consistent.

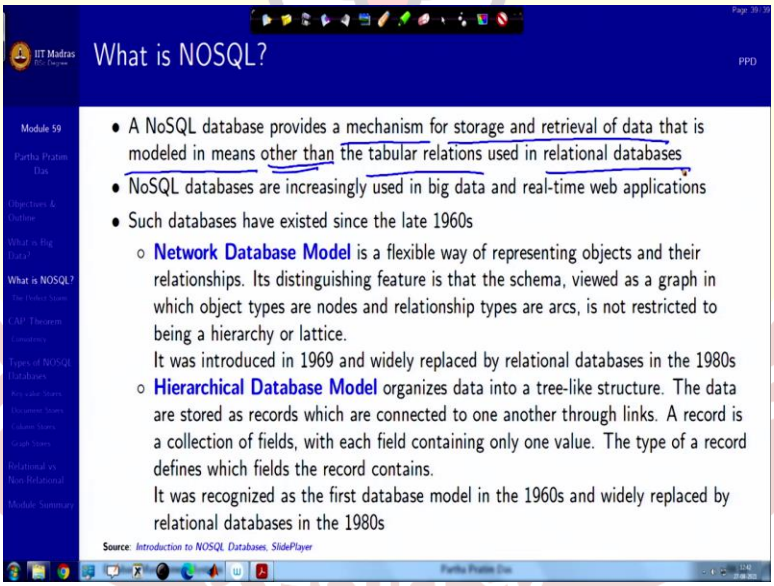
The veracity, the quality of capture data can vary greatly, some data are very accurate, some are, are of very low accuracy and so on. So, these all, these five together all starting with V is known as the 5V's or characteristics of big data.

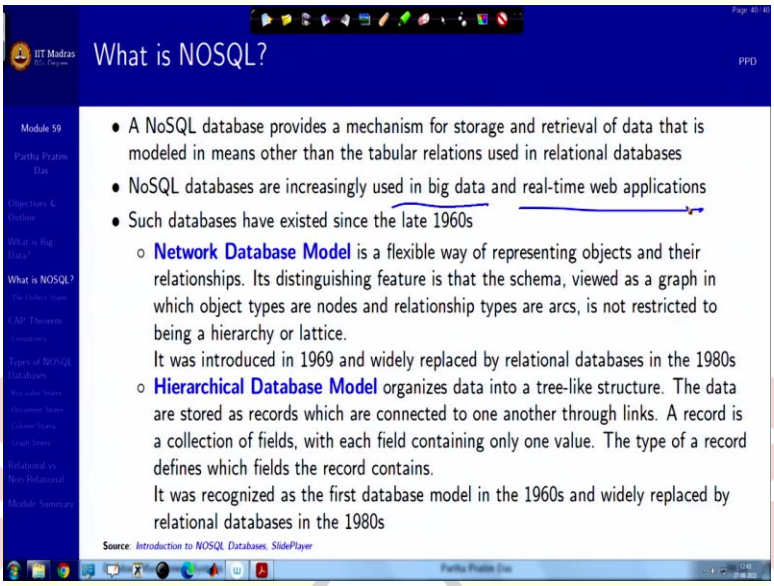
(Refer Slide Time: 06:03)

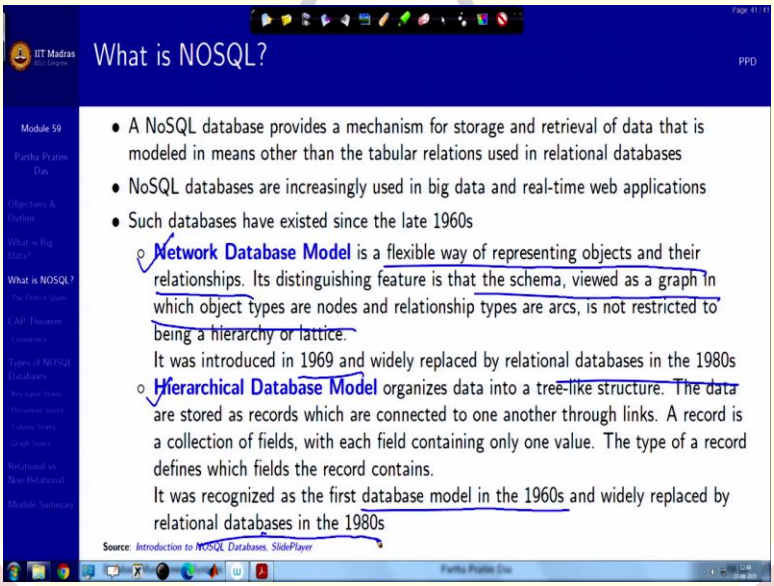


So, the challenge here basically is, how do you deal with applications database systems, which will deal with such big data.

(Refer Slide Time: 06:07)







So, NOSQL is a kind of mechanism, is not a particular database system is a kind of mechanism for storage and retrieval of data, which is what was the objective of relational as well. That is, modeled in means of, in means other than tabular relations used in relational database. So, you try to model them differently, tabular relations do not cater to easy management or feasible management of such scales of data in terms of storage and retrieval.

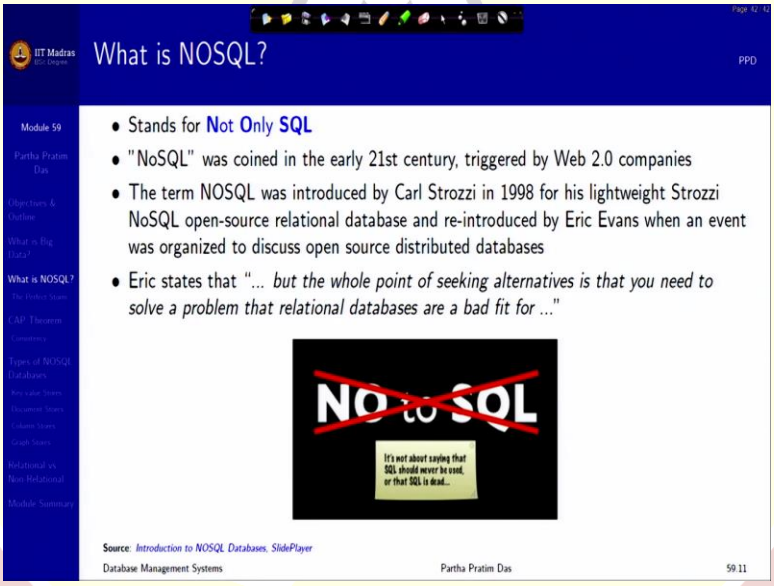
So, in the in the last module, if you recall, when I started talking about the performance and scalability of RDBMS I talked about alternate models. So, these are the alternate models that we are talking about in NOSQL. Now, this is not a surprise, because you are having big data and

you are having real time web application, web 2.0 you need all of that. And interesting thing is, actually all this started long ago almost actually before the relational data models came in.

So, you had hierarchical data model, which was a first formally accepted data model from IBM in the 1960s and network data model. So, hierarchical is a tree kind of a structure and which links the data records one to the other. And a network is more like a graph kind of a model. So, it is, the view the schema is viewed as graph of objects and their nodes.

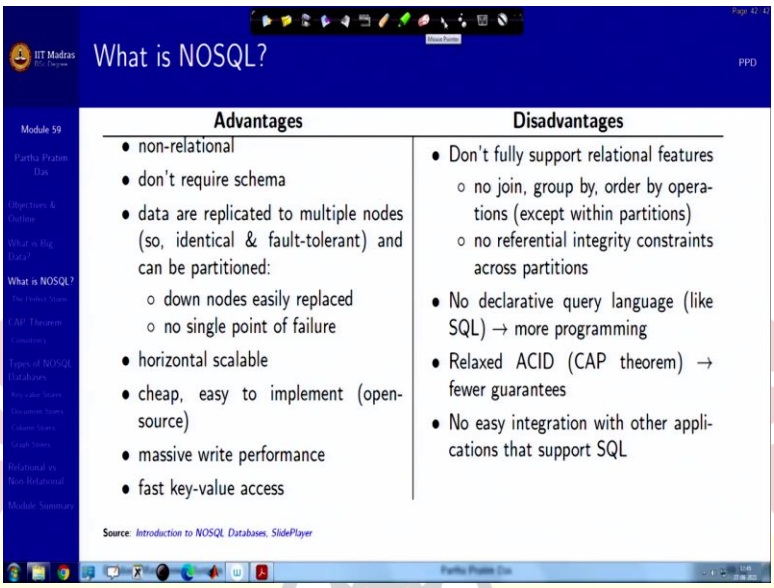
And these two came in, early 70s. And both of them actually survived actively till relational databases, which came in later part of the 70s, started dominating the whole field. But now, these or basically evolutions of these are making a comeback in terms of the NOSQL.

(Refer Slide Time: 08:10)



So, what does NOSQL means? NOSQL is not, no to SQL, NOSQL is just too powerful. It is just too useful for several enterprise applications. It stands for not only SQL, so it is, it is a lot more than that. So, this, this coinage came strongly in the 21st century, it was first used by Carl Strozzi in 1998. But then it was reintroduced by Eric Evans, who did a lot of work on NOSQL and that has become a term for these kinds of databases.

(Refer Slide Time: 08:44)



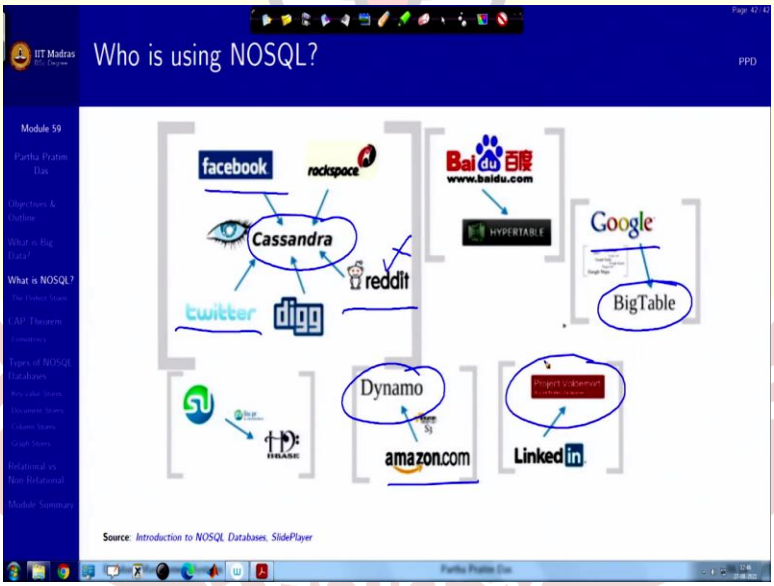
Now, there are several advantages, comparative advantages and disadvantages with respect to the relational. It is not relational; it does not require a schema to be frozen beforehand. Data is replicated to multiple sites in relational, you will remember how strongly we fought to reduce redundancy. If you do not do that, you do it differently. And the data can be partitioned. If a node is down, it can be easily replaced, there is no single point of failure.

It is horizontally scalable, scale out we talked of. It is cheap, easy to implement, massive performance in right first key value access and so on. In comparison, you have all the known properties of RDBMS, which do not compare to them. Like, the disadvantages would of NOSQL will lead to the fact that you do not have a declarative programming language. So, you need to do more of programming. The ACID is compromised, because otherwise you cannot work with the 5V's. It does not support the relational features fully, maybe partially. There is no easy integration process and so on. And we will see through some of these in the later part.

(Refer Slide Time: 09:57)

So, I am talking about some of the well-known common popular NOSQL databases are Cassandra, MongoDB, your H base, Dynamo, all of that.

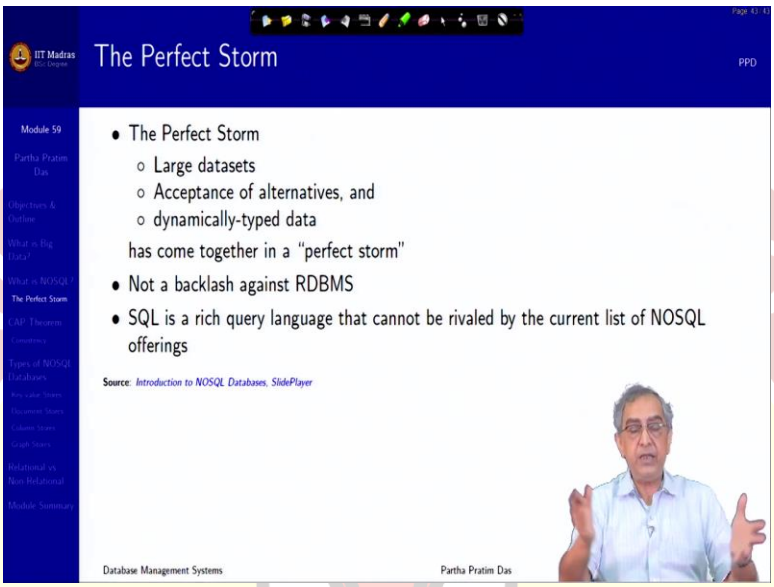
(Refer Slide Time: 10:16)



And this is just a glimpse of what some of our leading companies are using, I mean web application, web 2.0 companies are using. So, you can see the Facebook, which uses Cassandra. Twitter also uses that, Reddit also uses that, your Google use their own Big Table, which is one of the initial points of NOSQL. Amazon uses their own Dynamo, and LinkedIn use this

particular NOSQL database and so on. I mean, there could be several, I just wanted to give you a glimpse of where, where we are in terms of this.

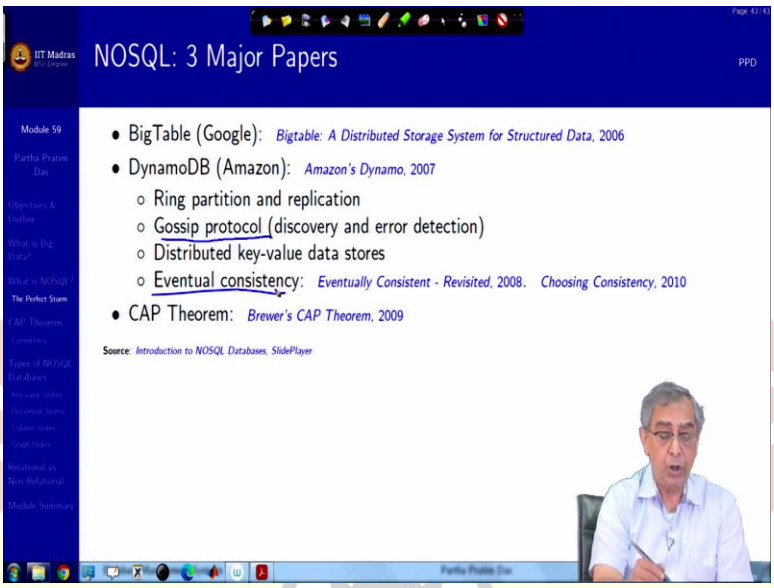
(Refer Slide Time: 10:57)



So, as it is called that, how did NOSQL happened? So, it is called the perfect storm. That is something we change a lot of things to support large datasets. Accept alternatives, like do not be so rigid, mathematically and otherwise, in as you are in the relational. And accept dynamically typed data, these three concepts came together to create a perfect storm in the world of web applications, in the world of database application.

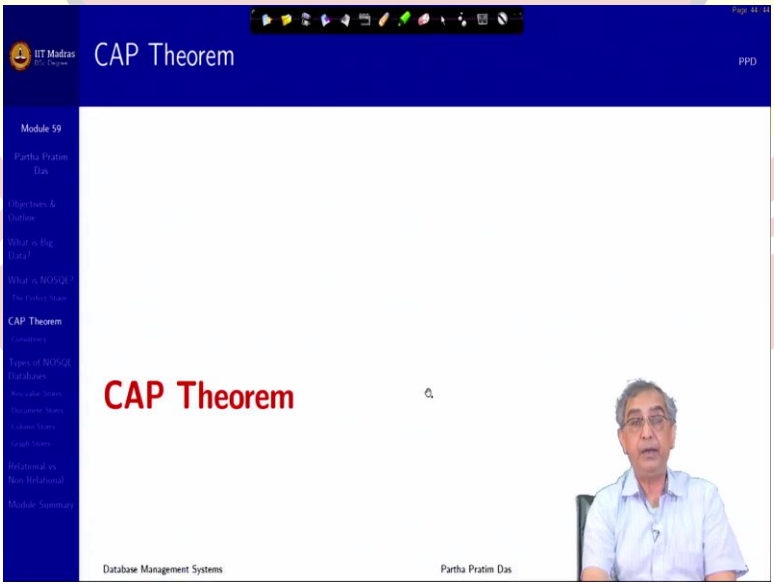
It has to be remembered that this is not a backlash against RDBMS. The question is not, that RDBMS is no more useful, but it is. For, for example, you are doing a net banking transaction on net, you will not use a NOSQL. Because of the compromised guarantees that, NOSQL databases will normally have. SQL is a rich query language. And it cannot be rivaled by any of the current NOSQL offerings. But it is an, it is a different space of big data, it is a different space of web 2.0 applications where you need this kind of new database models.

(Refer Slide Time: 12:15)



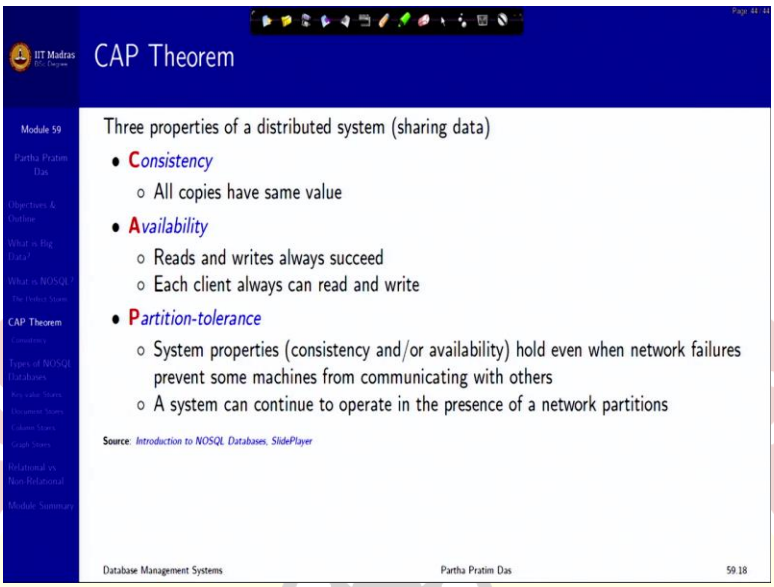
So, it all started in the, in around 2006 to 2010, that time period. So, it is about 12, 15 years back is when there were three pioneering papers. One by Google, called Big Table, one by Amazon called Amazon's Dynamo. And one by Eric Brewer, the CAP theorem which we will study. And it came with a lot of interesting concepts, that after the CAP theorem, some of them are just to briefly talk about that is gossip protocol and the eventual consistency concept.

(Refer Slide Time: 12:56)



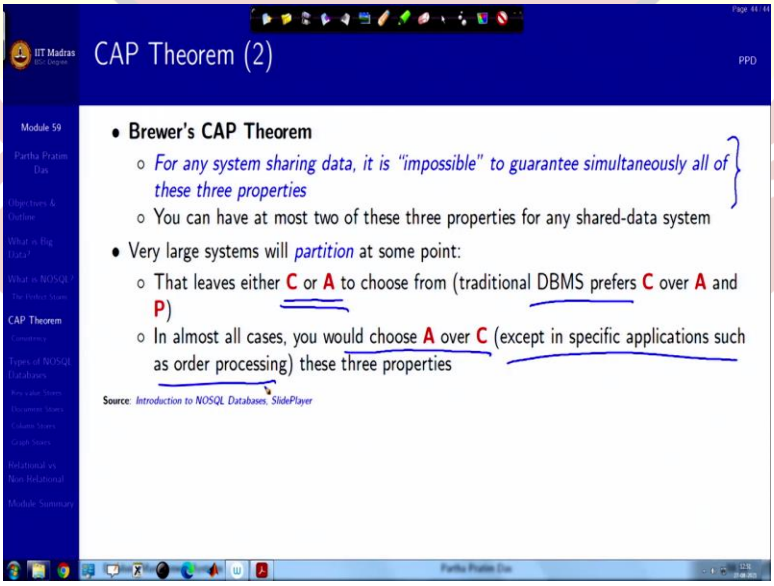
So, let us see what the CAP theorem talks about.

(Refer Slide Time: 12:57)



CAP theorem is kind of the equivalent of ACID in terms of the NOSQL. So, C stands for consistency. That this data is heavily replicated all copies of the same value is what you want to achieve. The avail A is for availability, that is read and write always succeed. That is each client always can read and write, that should happen. And it is partition tolerance, that is a system can continue to operate in the presence of network partition, the database being divided. So, these three properties together is known as the CAP theorem.

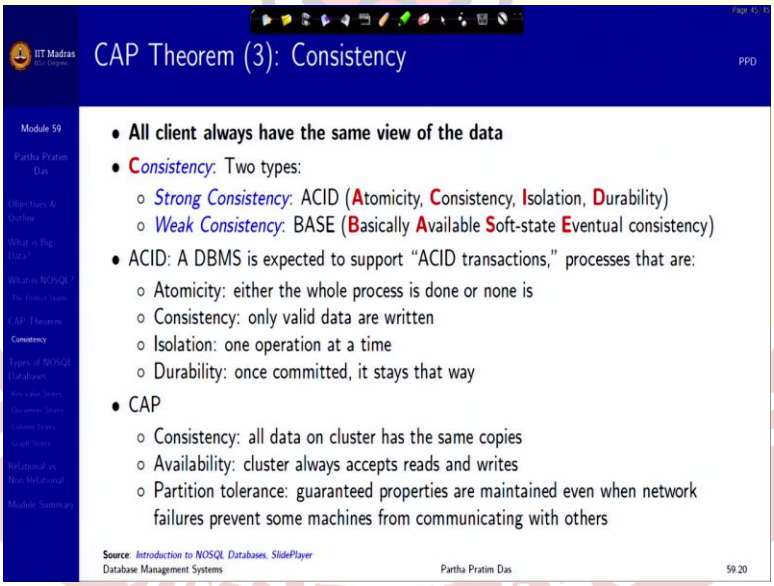
(Refer Slide Time: 13:36)



Now known as CAP, the CAP theorem says this is very, very interesting. That for any system sharing data, it is impossible to guarantee simultaneously all of the three properties. You cannot have all of them together, you can have only two at a point. Now, so, why is it called a theorem, there is no proof for that. But there is no counter example either. So, this is more like, it is called a theorem but it is more like a realization, more like a principle.

So, since you are talking about very large systems, they will partition at some point of time. You cannot easily live with systems which will not partition. So, that will leave you with the choice of which of the two, between C or A you will choose from. Now, if we talk about traditional databases, then we prefer C. That is, well at some points the availability may be low. The system may be busy, not allowing me to access. The partition may not be possible, but consistency will have to be there. Now, here in many cases, we choose A over C, that is availability over consistency. Of course, not in such applications. So, they will go into RDBMS, they will not come for NOSQL.

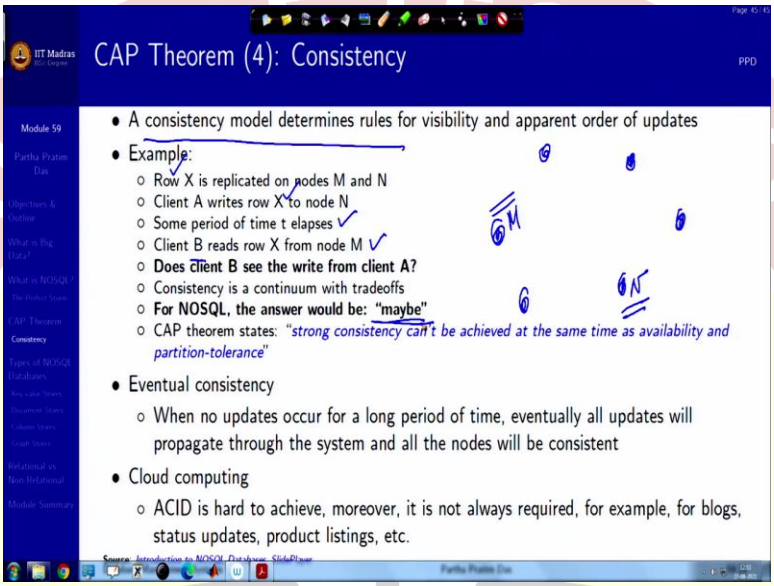
(Refer Slide Time: 15:11)

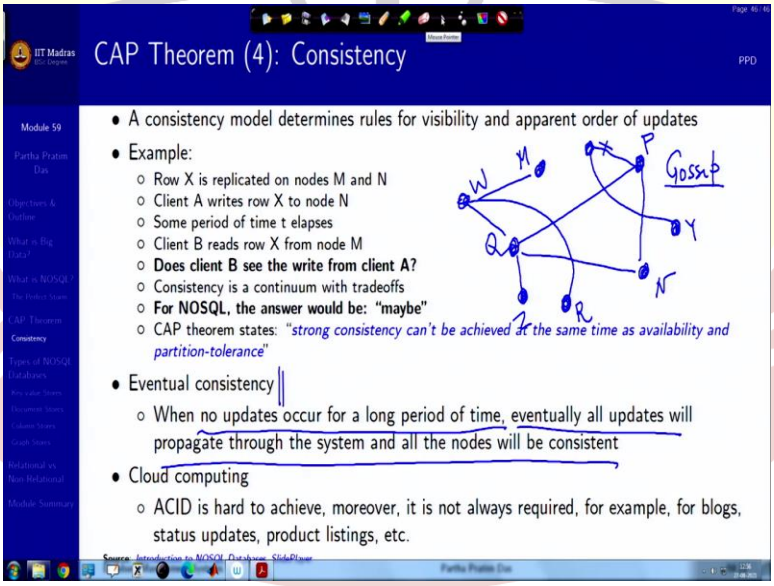


So, what does this consistency means? Consistency says that all clients always have the same view of the data. Now, the where it differs from what we studied in the RDBMS is that RDBMS follows a strong consistency, the ACID properties, atomicity, consistency, isolation, durability you know it very well.

Whereas, now, we are talking about a weak consistency which is nicely acronymed as BASE. Basically available soft state eventual consistency, basically available. So, availability is the first thing. Soft state, that is it is not very hard and fast. And it will be eventually consistent, not every time consistent.

(Refer Slide Time: 16:03)





So, given this, let us look at a, look at an example to understand what we mean by this consistency. So, the consistency model, determines the rules of visibility and the order of updates. Remember, there are multiple nodes in the system, which hold the data. And they are,

they are maybe, they may be weakly connected, not they are eventually connect, they are necessarily connected but all may not be connected to everyone else because that is not possible.

So, when we, you say row X is replicated on nodes, M and N, these two nodes. So, client a, so, row X is available on nodes M and N. So, client a writes row X to node N here. So, after some point of time, so, what will happen? The client is written to node N, so it will be available at node N what, what the client has written. But not on node M. After some period of time T elapses, some random period of time T. The client B reads X from note M.

The question is, does client B see what was written by A. The consistency is a continuum with trade off. In relational model, we will say that, of course, otherwise the data is inconsistent. Here it is not, in SQL, the answer is maybe, maybe if node N has been updated on node M, otherwise not. So, what you do is, instead of trying to I am sorry, I, erase this. Instead of updating all these nodes at the, at the same point, if this is updated, then you follow, I am just very, at a top level, I am just trying to give you the philosophy.

What you do? You do a gossip, that is node N, after a random amount of time, sends this updates, this data on a random set of nodes, say. So, at this point, if some client reads the data from N P or Q, it will get the updated data, someone reads from M or R gets a not updated data, it is maybe. But the gossip continues. So, after a random amount of time, Q, let us, let us have some more. This is X, this is Y, this is Z, this is W.

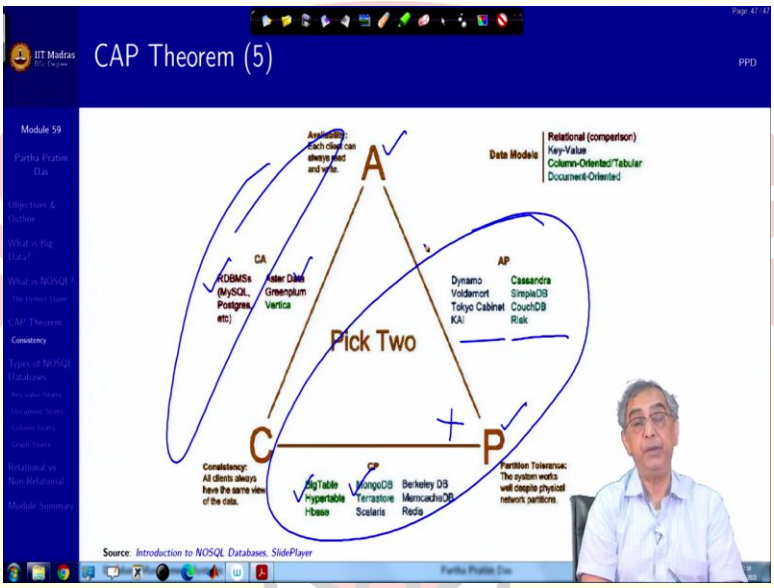
After some time, random time, Q updates the data on some nodes randomly, maybe tries to update on P also. After some time, P updates on X. After some time, W updates on R and updates on M. After some random time, X updates on Y. So, the process is very simple. It is the way the gossip and why is it called a gossip protocol? Because it is a way that gossip propagates, that you hear something, you randomly tell a few of your friends.

You do not tell everybody. Few of your friends randomly tell their friends. Some of them, randomly tell their friends, then eventually over a period of time all your friends get to know that gossip. So, it is that same kind of a protocol. Of course as balancing of, how much random delay you give between that gossip cycles, how many nodes randomly you would send this write to.

But over a certain period, if no updates occur for a long period of time, eventually all updates will propagate through the system and all nodes will be consistent. This is the concept of

eventual consistency. That is what the gossip protocol supports and that is what is the basis of, the BASE properties in contrast to the ACID properties. And this relaxation makes the feasibility of NOSQL systems much better.

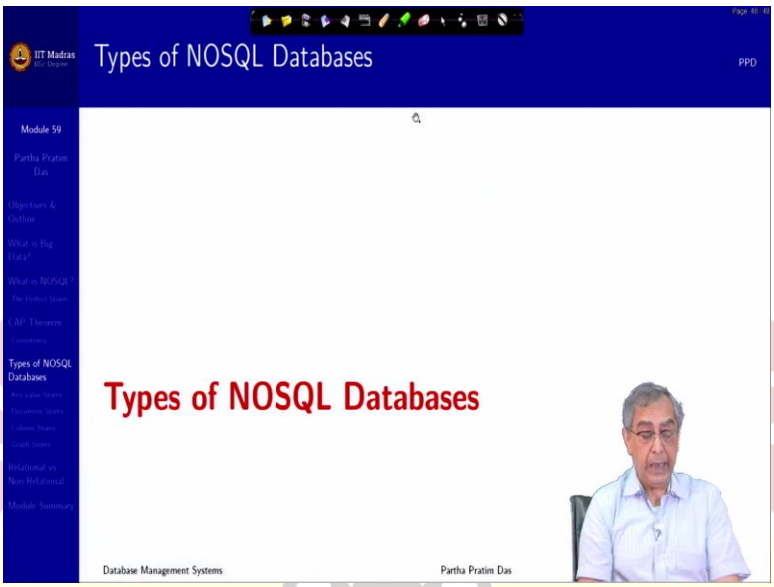
(Refer Slide Time: 20:48)



Yeah, again some examples for you, to appreciate that what is the consequence of CAP's theorem that at most two can be simultaneously guaranteed. So, for example, Cassandra and Dynamo these prefer partition and availability, which is absolutely understandable. And therefore, you have certain inconsistency, eventual consistency rather in the system. Whereas, Big Table, Hyper Table, MongoDB, they prefer consistency and partition.

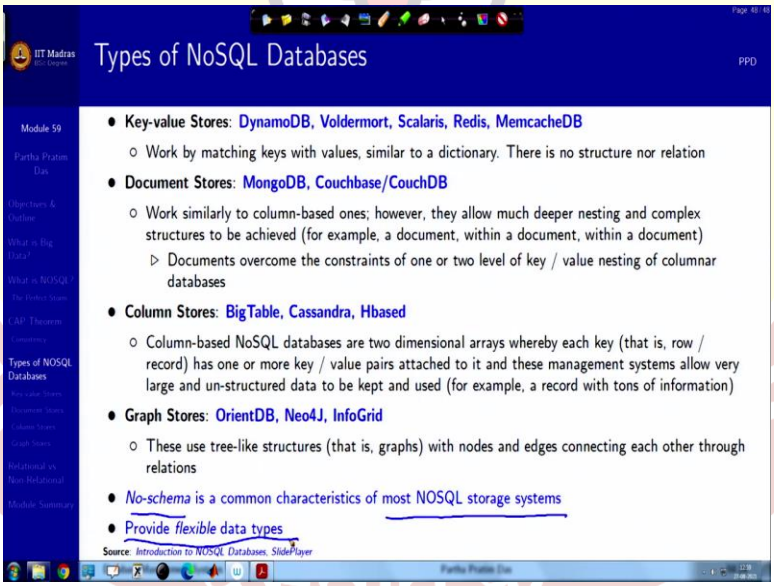
So, availability may be low. And there are, there is RDBMS, particularly. Which do not consider partition, that is not the priority, but the priority is availability and consistency, the ACID properties. But there are, there are several. So, this side, primarily CA gives you the ACID and leads you to the relational database systems primarily and other database systems, which has a similar property, not every database system, every non NOSQL database system is also relational. But most of the NOSQL systems fall under AP or CP categories. So, that is the beauty of the whole system.

(Refer Slide Time: 22:17)



So, let me just quickly, this will be just a quick, this is not for you to really, understand every bit or remember every bit, but just to give you a quick view.

(Refer Slide Time: 22:28)

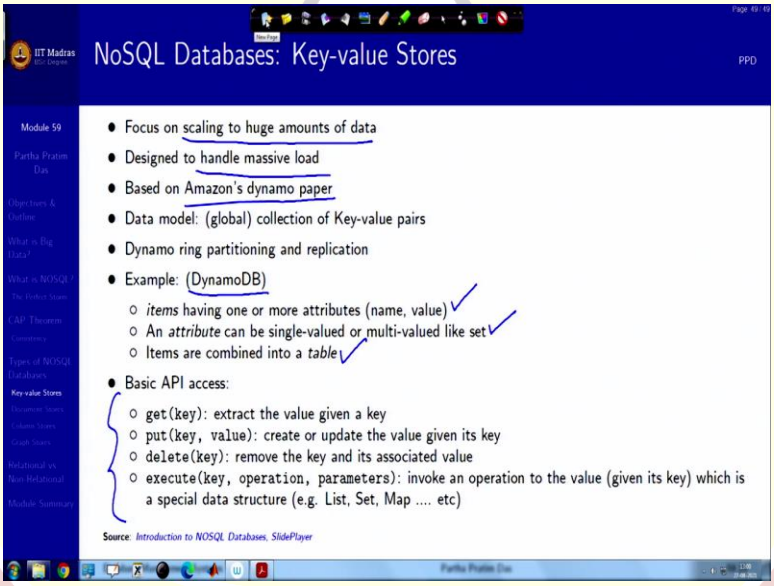


That NOSQL, as I said, is not one model now, not like relational, there are key value stores, which keep key value pair, there are databases, NOSQL databases, which is primarily store documents. Some are organized in terms of columns and super columns, and some work primarily on graphs. So, what is common between all of them, these are the four dominant types

of NOSQL databases that you have today. The dominant common properties, none of them have a schema.

It evolves as the data comes into the system. So, there is a paradigm shift from what you were doing in the relational database. When we had first analyzed and thought about the schema set, that schema is frozen in stone, you cannot change that, once your database has data in it. But here you do the reverse, you say there is no schema, the schema will keep on evolving as the data comes. So, it is a common characteristic of most NOSQL storage system. So, it provides a very flexible set of data types, even the data types can be increased in the life of the system. Very different kinds of.

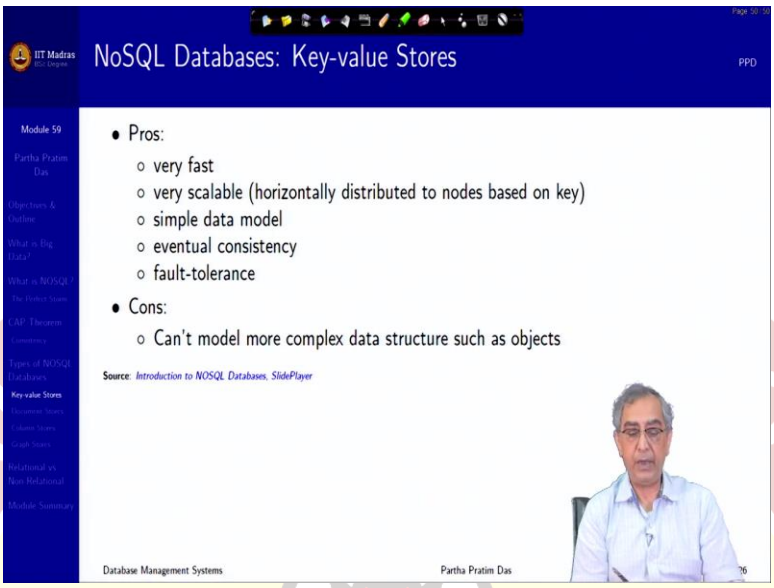
(Refer Slide Time: 23:45)



So, quick look into a few. For example, key value store, focus on scaling to huge amounts of data, can take handle massive load. Amazon's Dynamo, based on Amazon's Dynamo paper DynamoDB. So, items have one or more attributes, simple. Attributes can be single valued or multiple valued. Items are combined into tables. But they are not relational. They do not go through all those constraints and all those dependencies and all of that.

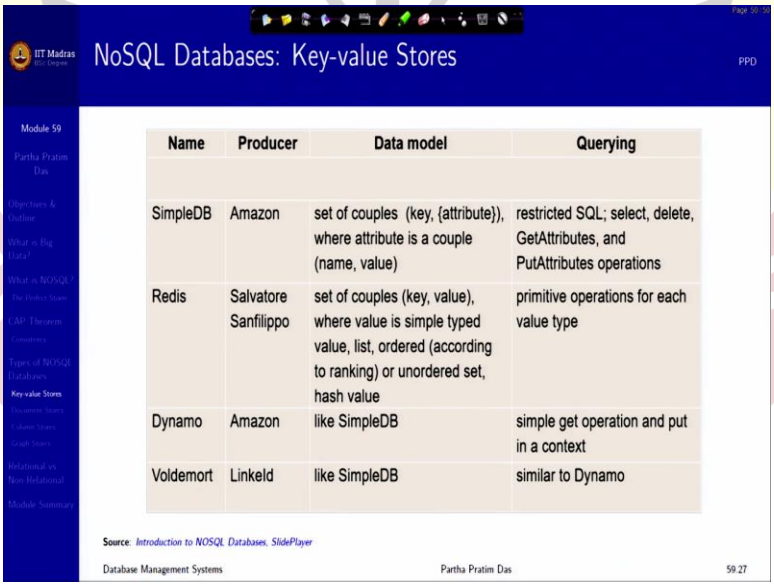
But it is basically key value pair. So, the basic APIs will have, get a key, extract the value given a key, put a key that is right, delete a key, this is remove or execute. If that could be operations associated with these nodes and you can execute them, like you can list map this kind of.

(Refer Slide Time: 24:41)



So, it is very fast, very scalable, simple data model, eventually consistent, fault tolerant, but it cannot model more complex data structure like objects. Because we are having a key value pair only. So, many use the, this kind of, key value stores as stores non NOSQL database, when these requirements are primary and complexitize, it is not a primary case.

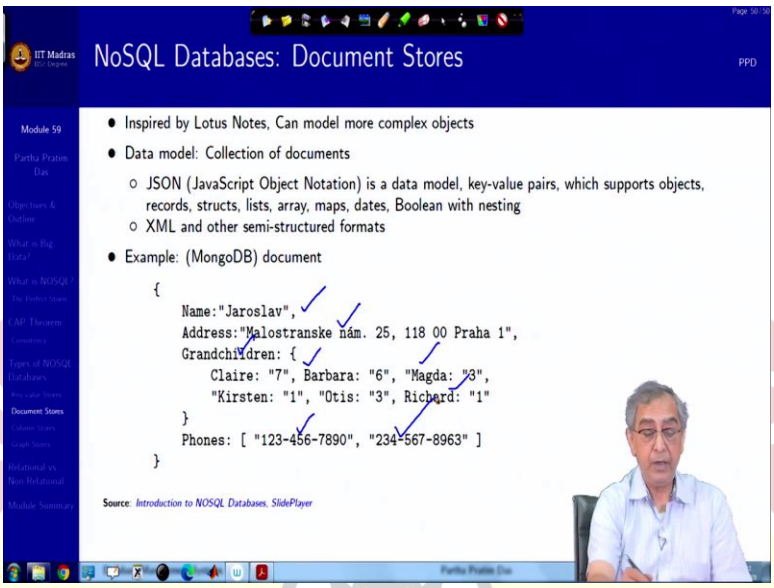
(Refer Slide Time: 25:15)



Name	Producer	Data model	Querying
SimpleDB	Amazon	set of couples (key, {attribute}), where attribute is a couple (name, value)	restricted SQL; select, delete, GetAttributes, and PutAttributes operations
Redis	Salvatore Sanfilippo	set of couples (key, value), where value is simple typed value, list, ordered (according to ranking) or unordered set, hash value	primitive operations for each value type
Dynamo	Amazon	like SimpleDB	simple get operation and put in a context
Voldemort	LinkeId	like SimpleDB	similar to Dynamo

So, these are some of the examples, I will not run through the basic properties. But I have given the name, the producer of who actually produces that database, what is the, what is the kind of data model that eventually gets used. And what is the different querying that exists.

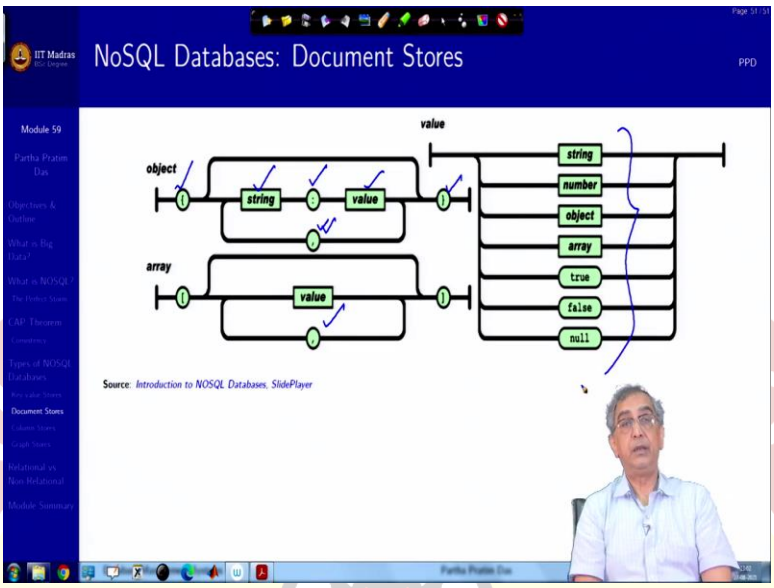
(Refer Slide Time: 25:33)



The second is document store, which is, which tries to keep everything as if it is a document, the data is as if it is a document. This was again, inspired by a very, very old model called Lotus Notes, which used to be there possibly in 80s. And as a document, it can certainly model more complex objects, and it is a collection of documents. So, it can be, it can use XML or some semi structured format, can use JSON. And an example would be like this, that you write name.

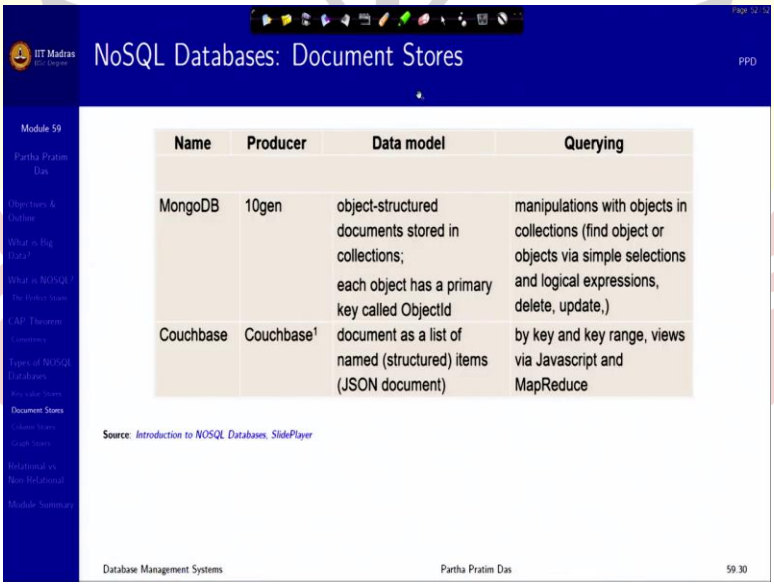
So, you can, you had learnt about XML tag and value pairs, so it is very similar. So, name is this addresses is this, grandchildren. Now, you are getting hierarchical, because you can have multiple values on this each one, then multivalued attributes like phone and so on. So, all this can be very easily stored in terms of document store structured.

(Refer Slide Time: 26:32)



This is the basic, structure of how it should be written. So, it starts with a curly brace, you have a string, colon value, which is the tag name. You can have multiple of them separated by comma and you end with again a closing brace. You can have an array; you can have different types of values.

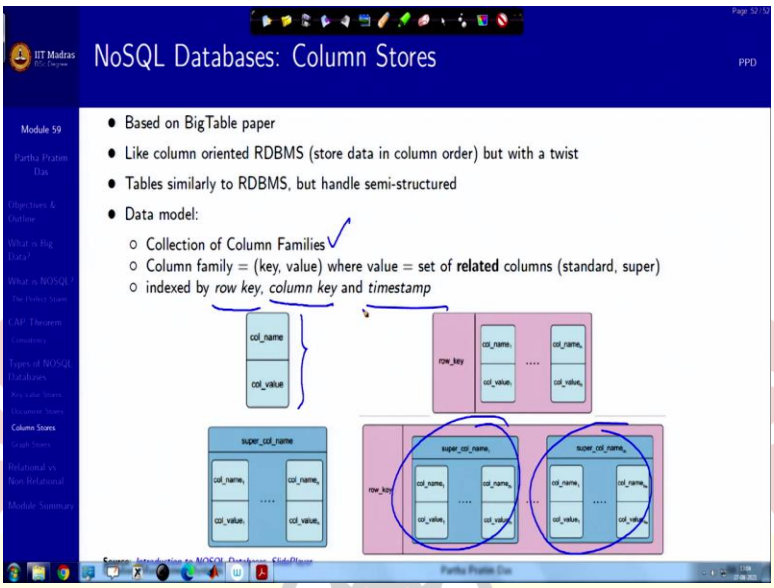
(Refer Slide Time: 26:57)



Name	Producer	Data model	Querying
MongoDB	10gen	object-structured documents stored in collections; each object has a primary key called ObjectId	manipulations with objects in collections (find object or objects via simple selections and logical expressions, delete, update,)
Couchbase	Couchbase[1]	document as a list of named (structured) items (JSON document)	by key and key range, views via Javascript and MapReduce

So, that is a document store NDB, MongoDB, Couchbase, both are very popular are the two different, at least two of the dominant documents store non SQL, NOSQL database system.

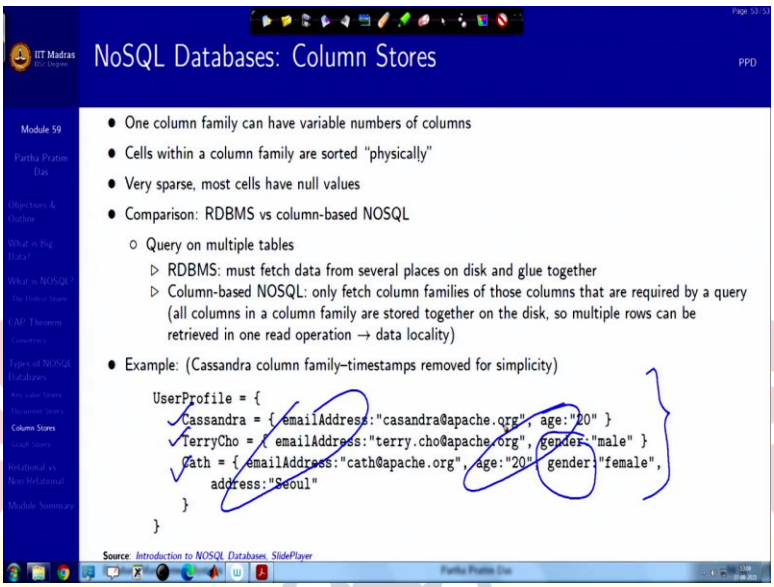
(Refer Slide Time: 27:15)



The third type is a column store, which is based on the Big Table paper of Google. So, it is like, it is column oriented like RDBMS but it is a twist, that it is that the handling is semi structured. So, what you have is not a table with pre decided set of columns. The columns go into collection of column families; a column family is a key value pair. And related set of columns can be built into super columns, columns of columns.

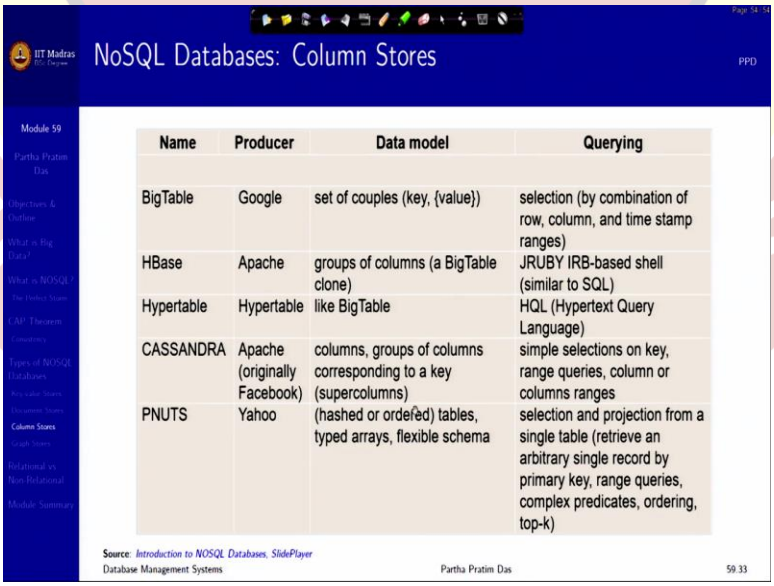
Hierarchical in a different way, is indexed by row key, column key and timestamp of where you have written. This timestamp is very important, you will recall, we have used time stamping in, in some of the protocols with reference to deadlock here. The timestamp would be important to guarantee eventual consistency.

(Refer Slide Time: 28:24)



So, one column family can have variable number of columns and cells within the column family are sorted physically. So, this is typically how the data will look. And you say this is the, these are the different entries in that. These are actually the columns, the age, gender, email address, but they are all flexible. You do not need to have all the column values for every item, that you have.

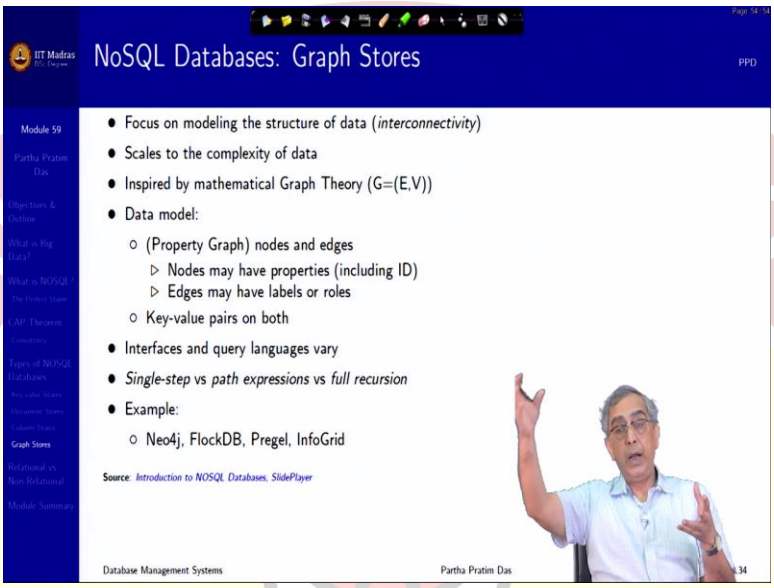
(Refer Slide Time: 29:01)



Name	Producer	Data model	Querying
BigTable	Google	set of couples (key, {value})	selection (by combination of row, column, and time stamp ranges)
HBase	Apache	groups of columns (a BigTable clone)	JRUBY IRB-based shell (similar to SQL)
Hypertable	Hypertable	like BigTable	HQL (Hypertext Query Language)
CASSANDRA	Apache (originally Facebook)	columns, groups of columns corresponding to a key (supercolumns)	simple selections on key, range queries, column or columns ranges
PNUTS	Yahoo	(hashed or ordered) tables, typed arrays, flexible schema	selection and projection from a single table (retrieve an arbitrary single record by primary key, range queries, complex predicates, ordering, top-k)

So, these are some of the NOSQL databases that support column store, Big Table of Google and Cassandra of Apache, originally Facebook had developed it which are very popular Hyper table also is quite popular.

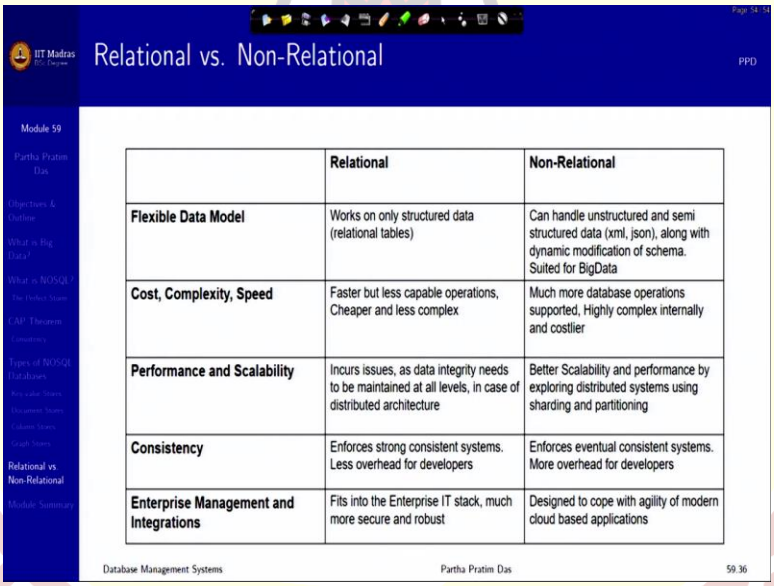
(Refer Slide Time: 29:19)



Finally, you have graph store, which basically tries to keep data as a graph. So, the data model is a property graph nodes and edges. Nodes may have a number of properties and edges may have labels or roles. So, kind of, you can see that this is probably an extension of an ER model that we have solved but the context is simply very different. The interface and the query languages vary across the graph structure.

It will, it may have a single step or path compression, full recursion because you have to naturally, since a graph you will have to traverse the graph to answer a query. So, these are some of the graph databases that you have.

(Refer Slide Time: 30:08)



	Relational	Non-Relational
Flexible Data Model	Works on only structured data (relational tables)	Can handle unstructured and semi structured data (xml, json), along with dynamic modification of schema. Suited for BigData
Cost, Complexity, Speed	Faster but less capable operations, Cheaper and less complex	Much more database operations supported, Highly complex internally and costlier
Performance and Scalability	Incurs issues, as data integrity needs to be maintained at all levels, in case of distributed architecture	Better Scalability and performance by exploring distributed systems using sharding and partitioning
Consistency	Enforces strong consistent systems. Less overhead for developers	Enforces eventual consistent systems. More overhead for developers
Enterprise Management and Integrations	Fits into the Enterprise IT stack, much more secure and robust	Designed to cope with agility of modern cloud based applications

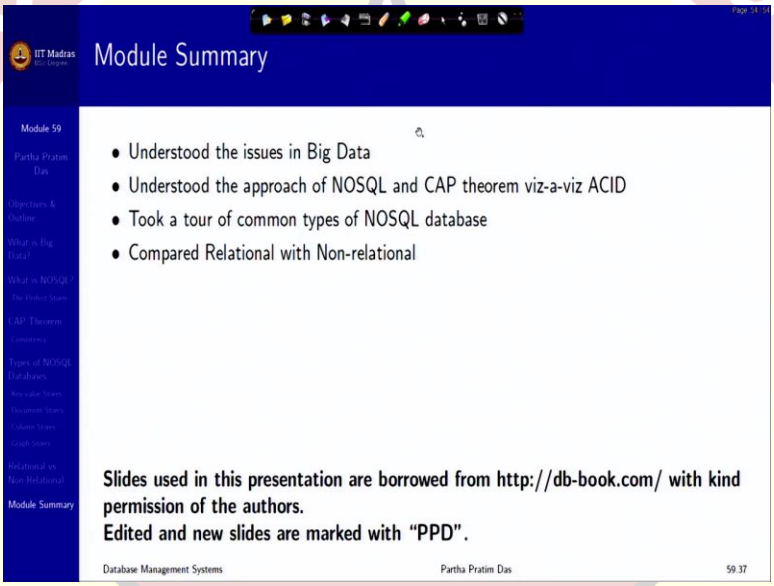
Now, finally, before we conclude, let us summarize by taking a look into what did we achieve by getting non-relational or getting into NOSQL. We have in terms of flexibility of data model, relational works only unstructured data, non-relational can handle unstructured, semi structured data. Along with dynamic modification of schema, which are very suited for big data. The cost complexity and speed relational of course, is faster for less capable, I mean less complex operation, cheaper but less complex.

But non-relational, NOSQL is much more database operations are supported, highly complex internally and is costlier of course, performance and scalability. It in, we have seen specifically

that it incurs issues data integrity needs to be maintained at all levels. So, getting to a case of distributed architecture was a problem. And NOSQL has better scalability performance. And by changing from ACID to BASE kind of expectation.

Consistency is enforced everywhere in relational, eventual consistency is what you look for in non-relational. Enterprise management and integration would primarily be relational, because it fits into the enterprise's IT stack. But non-relational is designed to cope with ability to agility to manage modern cloud based applications web 2.0 and onwards.

(Refer Slide Time: 31:56)



So, that was what I wanted to give you a glimpse of what is happening in the database spaces, in the recent times, in the last about one to two decades. So, we have talked about big data, NOSQL, the differences that NOSQL is designed with. And discussed four common types of NOSQL database systems. Thank you all very much for your attention, and we will meet in the next module, the concluding module of the course.